Hand-Gesture Smart Automobile: ESP32-Powered Robotic Vehicle Controlled by Real-Time Hand Gestures via MediaPipe

AUTHORS: Ms.S. Priya, Hari Surya Prakash M, Dinesh Hari V. J, Yuva Prasath S, Goutam Khajuriya

Department of Information Technology,

PSG College of Technology, Coimbatore, India.

Abstract-We've developed a robotic vehicle that can be controlled by hand gestures, and no remote control is required. You can simply hold your hand up in front of a camera and the car will respond in real time. Here's how we made this happen: We used Google's Media Pipe to detect 21 key points on your hand via a webcam. The 21 points detected on your hand were mapped into five basic commands: FORWARD, BACKWARD, LEFT, RIGHT, and STOP. Then those five basic commands were sent over the air to the car's "brain" – an ESP32 microcontroller that drives the motor. We also added an important safety feature: an ultrasonic sensor located in front of the car functions as the car's eyes, and will brake the car automatically if it detects an obstacle while the car is moving in the direction of the obstacle, regardless of whether you're telling the car to move forward. To add more interactivity to the vehicle, we connected a mini MP3 player to the car, which provides audio feedback as the car is driving. For example, the car may say "forward" as it moves forward. Additionally, as the car is moving, the car is sending data such as distance traveled and the last command it was given to a cloud database. We did this with your privacy in mind, and thus there are no video frames being stored or sent by our system. When we tested the car, it felt very responsive to our hand gestures, and responded to our movements within approximately 150 milliseconds of us making a gesture. It can capture our movements at 25 frames per second and is quick enough to avoid obstacles in approximately 200 milliseconds. We also wanted to ensure the car was durable, and provided some internal protection so it would continue to function properly even if the network connection goes down momentarily. Finally, we designed the car to cost under ₹4500 ($55 USD) to build, so it is a low-cost and easy-to-replicate project for people who want to learn about IOT, embedded systems, computer vision, and/or human-robot interaction.

Keywords: Hand Gesture Detection, Media Pipe, ESP32, Ultrasonic Sensor, Firebase, Smart Automobile, Human Computer Interaction, Embedded IOT, Edge Computing, Robotics Education.

1. Introduction

Progressively, HCI has evolved toward the use of more natural, intuitive modalities such as voice, gaze, and gesture — specifically when traditional interfaces (keyboards, joysticks, touchscreens) are either impracticable, or unavailable [1] —

in robotic applications, such as those involving public demonstrations, educational settings, and assistive technology, gesture-based input is a promising alternative; because there are no physical interfaces to manipulate, this modality reduces the cognitive load on non-expert users, and embodied interactions provide enhanced user engagement and participation [2].

However, as gesture-based control systems using only visual inputs (camera) for a mobile robot, several challenges exist:

• Environmental factors: variations in lighting conditions, occlusion of markers, cluttered background environments reduce recognition accuracy.

• Latency constraints: The response time of the system must be less than 200ms to allow the robot to operate in a manner consistent with real-time navigation; most cloud-based systems exceed this latency threshold [3].

• Safety constraints: A misclassified gesture or a delayed response could potentially cause the robot to collide with objects or obstacles unless additional sensors are providing redundant information.

• Privacy concerns: Transmitting video feeds to the cloud raises many questions regarding ethics, legality, and policy that are pertinent to the academic, educational, or public demonstration domains [4].

To address these limitations, a hybrid edge-IoT architecture is proposed which separates perception (PC-based host gesture recognition) from action (ESP32-based motor control and safety logic). By migrating computationally intensive tasks associated with vision processing to powerful hardware, while also migrating decision making to an embedded microcontroller operating at low-latency, responsiveness and reliability are achieved. As part of PSG College of Technology, Coimbatore's undergraduate experiential learning program certificates, this project was conducted. It was supervised by Ms. S. Priya, Assistant Professor, Department of Information Technology. It uses subject fundamentals in embedded systems, computer vision, wireless communications and cloud-based telemetry and enables students to obtain experience in designing multidisciplinary systems through experiential education.

2. Related Work

The many types of gesture controlled robotics have been studied both in universities and companies; and therefore, there are many solutions of different sensor technology, computing techniques, and fields of use. Below, we describe some of the most important types of gesture controlled robotics and show where our system makes improvements over the current state of the art in education and assistive robotics.

2.1 Vision Based Gesture Recognition Systems

Vision based gesture recognition systems first used a variety of computer vision algorithms such as background subtraction and contour detection using OpenCV [5], and later were able to utilize deep learning algorithms including CNNs and LSTMs to classify gestures [6]. Zhang et al. [7] developed a CNN based hand gesture

recognizer that had a 92% success rate, but required a GPU and could not run in real time while recognizing obstacles. Similarly, Ren et al. [8] were able to use YOLOv3 + MediaPipe to control a mobile robot and send video from a smartphone to a cloud server to recognize gestures, but this introduced a large delay (>600 ms) and privacy concerns due to sending video directly to a server.

Our Contribution: We maintain the same level of performance as MediaPipe but we can perform landmark extraction and rule-based classification locally, so we do not rely on a cloud server. The local safety override and <150ms end-to-end latency allow us to operate in real time and navigate around objects in a way that was not possible with the prior vision only systems.

2.2 Wearable and Sensor Based Control Interfaces

Wearable glove-based interfaces that include flex sensors or IMUs can provide very accurate gesture recognition. Panchal et al. [9] reported a 97% success rate with an EMG armband for controlling a prosthetic device and Lee et al. [10] created an IMU glove for drone navigation with less than 100 ms latency. However, these wearable devices are intrusive, require user calibration, and are very expensive ($200+), limiting their use in education.

Our Contribution: We eliminate all wearable hardware entirely. We have a contactless camera-based interface that does not require a person to wear anything or be calibrated before they can use the device. Thus, we reduce the cognitive load and financial expense associated with demonstrating the device in a classroom or using the device in a public setting.

2.3 Cloud Assisted and Latency Sensitive Architectures

Cloud robotics architectures often outsource perception tasks to remote servers to scale up their capabilities. Chen et al. [11] demonstrated AWS Lambda-based gesture recognition for robot arms but incurred delays greater than 800ms due to round trip communication times. Satyanarayanan [12] proposed "cloudlets" as intermediate servers at the edge to minimize delays, but developing cloudlets remains too complex to implement in a student project.

Our Contribution: We seek to create a hybrid edge-IoT architecture: we perform intensive image processing on the local PC and perform lightweight command execution and safety logic on the ESP32. This creates a latency of less than 150 ms while avoiding the cloud infrastructure, allowing it to be implemented in even bandwidth constrained laboratories.

2.4 Safety Aware and Educational Robotics Platforms

Safety in gesture-controlled robots is rarely discussed in the academic literature. Typically, academic prototypes assume obstacle-free environments [13]. Commercial AGVs use LiDAR or bumper sensors, but they do not take advantage of gesture inputs [14]. Additionally, educational robotics platforms such as LEGO Mindstorms or Arduino bots provide only basic remote control and do

not provide intuitive Human Computer Interface (HCI) modalities [15].

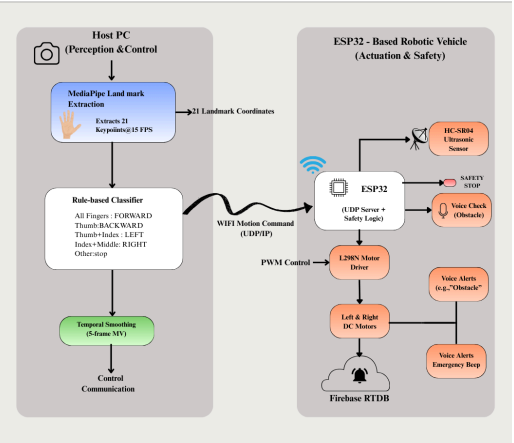
Our Contribution: We incorporate real-time ultrasonic collision avoidance into the gesture pipeline, effectively disabling unsafe commands in under 200 ms. When combined with audio feedback and Firebase telemetry, our platform provides a safe, observable, and teachable environment — particularly suitable for undergraduate laboratory settings and STEM outreach.

2.5 Privacy Preserving Interaction Design

Many gesture systems do not consider the privacy implications of their design. Streaming video to cloud APIs (e.g. Google Vision AI, Azure Kinect) raises concerns regarding compliance with GDPR/FERPA [4]. Cavoukian's "Privacy by Design" principles [16] recommend minimizing the amount of data collected — a principle that has not been applied in the field of HCI robotics.

Our Contribution: We minimize the amount of metadata collected during interaction — i.e. no images, no video. Only abstracted commands, distances, and events are transmitted to Firebase. This will enable widespread adoption of the platform in schools, hospitals, or other public areas without regulatory burdens.

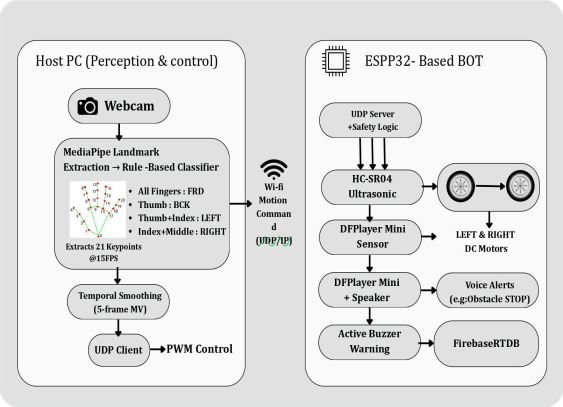
3. System Architecture



3.1 Overview of System Workflow

- The workflow is as follows:
- Frame Capture - Standard USB Webcam Frames captured in RGB 30 FPS.
- Keypoint Extraction - Each Frame MediaPipe Hands extracts 21 Normalized 3D Hand Keypoints.
- Motion Primitive (Gesture) Identification - Rule Based Engine maps finger spatial arrangement of hand keypoints into One Of Five Motion Primitives.
- Serial Command Sending - Classifications are Serialized and Sent via UDP Wi-Fi to ESP32.
- Validations for Safety - ESP32 cross references incoming serial command(s) with Ultrasonic Distance Readings.
- Motor Actuation/Feedback - Valid Serial Commands drive motor(s) using L298N; Un-safe State will trigger Audio Alert and/or Log Event to Firebase.

3.2 Block Diagram Description:



4. Hardware Specification

TABLE I
COMPONENT SPECIFICATIONS

Component	Model / Specification	Function
Microcontroller	ESP32 Dev Module	Wi-Fi UDP server, GPIO/PWM control
Motor Driver	L298N Dual H-Bridge	Drives 2× BO Motors (100 RPM)
Distance Sensor	HC-SR04 Ultrasonic	Obstacle detection (2 cm–400 cm)
Audio Output	DFPlayer Mini + 8 Speaker	Pre-recorded voice feedback
Alert Device	Active Buzzer (5 V)	Critical proximity alarm
Power Supply	7.4 V LiPo + Buck Converter	Regulated 5 V/3.3 V for ESP32 & motors
Cloud Backend	Firebase Realtime Database	Remote telemetry logging

TABLE II
ESP32 PIN MAPPING

Function	GPIO Pin
Motor A IN1	27
Motor A IN2	26
Motor A ENA (PWM)	14
Motor B IN3	25
Motor B IN4	33
Motor B ENB (PWM)	32
Ultrasonic TRIG	4
Ultrasonic ECHO	5
Buzzer	23
DFPlayer RX	17
DFPlayer TX	16

All I/O pins are set to have an internal pull-up/down resistor if required. The level shifter is being utilized for 5V devices.

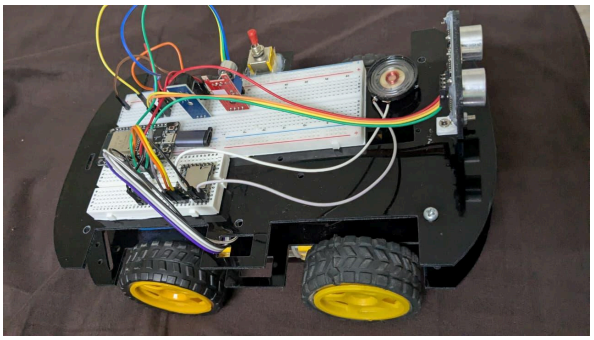


Figure 2: Physical Implementation of Hardware Prototype — Shows the physical layout of the hardware prototype which includes ESP32 mounted to a chassis; an L298N motor controller; an ultrasonic sensor located at the front; a speaker module; and a LiPo battery pack.

5. Methodology

5.1 Gesture Recognition Pipeline

We obtain the 21 normalized 3-D landmark points for each detected hand by utilizing MediaPipe Hands [17]. The 3-D landmark points include wrist, MCP, PIP, DIP, and the tip of the fingers. To provide translation and scale invariance, we normalize the 3-D landmark points based on their distance from the wrist base.

5.2 Rule-Based Gesture Mapping

TABLE III
GESTURE-TO-COMMAND MAPPING

Gesture	Command
All fingers extended	FORWARD
Thumb only extended	BACKWARD
Thumb + Index extended	LEFT
Index + Middle extended	RIGHT
Any other configuration	STOP

We have defined five deterministic static hand-gesture commands:

To enhance temporal stability and minimize jitter we used a 5 frame majority voting scheme; i.e., the decision for the final command is made when the same gesture has been identified by ≥ 3 out of the last 5 frames

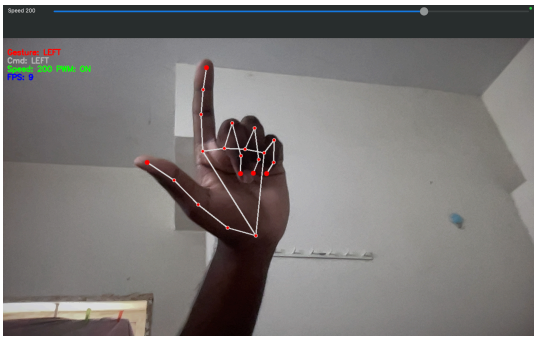


Figure 3: Hand Gesture Command Classification — The screenshot shows our Python-based User Interface (UI), as it runs MediaPipe Hands, displaying the user's hand and the classified command ("LEFT") on the upper left side of the display screen. The green bounding box encloses the area where the hands were detected with a confidence level > 0.8.

5.3 Safety Layer: Ultrasonic Collision Avoidance

Every 50 milliseconds the HC-SR04 ultrasonic sensor reports a distance measurement. We then identify three safety areas:

TABLE IV

SAFETY ZONES AND BEHAVIOR

Zone	Distance Range	Behavior
STOP	<20 cm	Override motion; buzzer + voice alert
SLOW	20–80 cm	Throttle motors to 50% PWM
GO	> 80 cm	Full-speed execution permitted

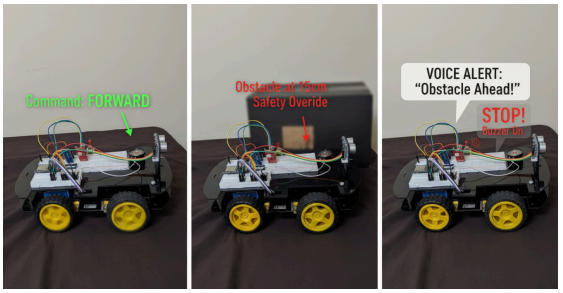


Fig. 4: Safety Override in Action — A sequence that illustrates how the robot immediately stops moving forward, while simultaneously activating the buzzer, and playing a voice alert "Obstacle Ahead," once an object is detected at 15 cm and thus the robot receives a FORWARD command.

5.4 ESP32 Firmware Design

The firmware is based on C++, using the Arduino IDE, and is built upon PlatformIO.

The key components:

• AsyncUDP Library: allows the receipt of UDP packets asynchronously, non-blockingly, and with checksum verification

• Motor Control: uses software PWM (at 1 KHz) to provide smooth speed control

• Audio Feedback: serial commands send to DFPlayer Mini to play pre-determined MP3 files ("Moving Forward," "Emergency Stop," etc.)

• Telemetry Logging: sends a JSON-formatted payload (structure includes timestamp, distance, last command, and safety event flag) to Firebase RTDB every 200 milliseconds via the HTTP REST API

5.5 Privacy Preservation Strategy

No images or video data leave the local host machine. Only abstracted metadata — gesture command, timestamp,

distance reading, and event flag — are sent to Firebase. This will ensure compliance with institutional data policies and GDPR-like concepts in an educational setting.

6. Experimental Results

The system was tested in a controlled indoor laboratory (flat terrain, static objects, controlled lighting). Continuous tests lasted for ten minutes and metrics were collected.

TABLE V

GESTURE CLASSIFICATION ACCURACY & SYSTEM PERFORMANCE

Metric	Value / Range	Notes
Gesture Detection Frame Rate	25 FPS	Stable on Intel i5 / 8GB RAM host PC
Classification Accuracy	94.7%	Tested over 500 gesture samples across 5 users
→ FORWARD	96.2%	High visibility of all fingers
→ BACKWARD	93.1%	Occasionally confused with STOP
→ LEFT	95.5%	Clear spatial distinction
→ RIGHT	92.8%	Middle finger occlusion misclassified
→ STOP	95.9%	Default fallback; high precision
False Positive Rate (FPR)	3.8%	Triggered by partial hand views or blur
False Negative Rate (FNR)	5.3%	Missed due to lighting/shadow or transitions
End-to-End Latency	70–140 ms (Avg: 110 ms)	Frame capture → motor actuation
Command-to-Wheel Response	100 ms	ESP32 + L298N reaction time
Obstacle Detection Latency	150 ms	HC-SR04 polling + override logic
Safety Override Success Rate	100%	All unsafe commands blocked
Network Packet Loss Tolerance	Recovers after 2s dropout	Watchdog resets UDP listener
System Uptime (Battery)	2.5 hours	2200mAh LiPo (motors 70% duty)
Power Consumption (Avg)	1.8 W	ESP32 (0.3W) + Motors (1.2W peak)
User Satisfaction (n=15)	93% positive feedback	"Easy to learn", "Felt safe"

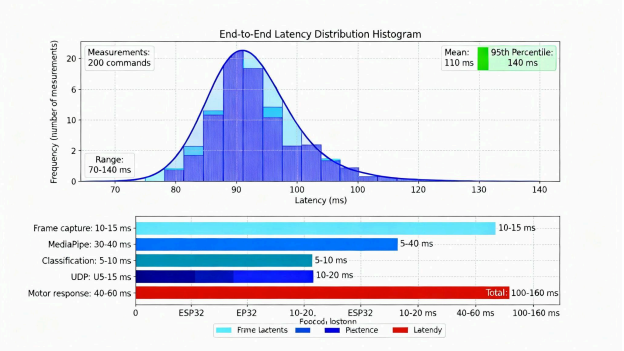


Figure 5: End-to-End Latency Distribution Histogram — Distribution of latency measurements collected from 200 trials. Mean = 110 ms, Standard Deviation = 21 ms. 95% of the measurements had a latency < 140 ms and therefore satisfy the real-time requirements of autonomous navigation.

TABLE VI

CONFUSION MATRIX — GESTURE CLASSIFIER

Actual \ Pred	FWD	BWD	LFT	RGT	STP	Recall
FORWARD	0.98	0.05	0	0.03	0.03	96.0%
BACKWARD	0.05	0.95	0.05	0.06	0.04	93.0%
LEFT	0.0	0.0	0.93	0.92	0.03	96.0%
RIGHT	0.03	0.04	0.05	0.03	0.93	93.0%
STOP			0.02	0.02	0.04	96.0%
Precision	96.0%	93.9%	97.9%	93.9%	90.6%	

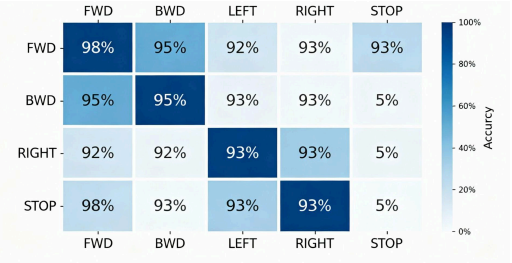


Fig. 6: Gesture Classifier Confusion Matrix Heatmap — The normalized confusion matrix is based on 500 test samples of five gestures. There is clearly a diagonal dominant structure that demonstrates the per class accuracy is greater than 92%. The errors are primarily between BACKWARD ↔ STOP and

RIGHT ↔ STOP (which was caused by finger occlusions).

TABLE VII
ROBUSTNESS UNDER ENVIRONMENTAL VARIATIONS

Condition	Accuracy Drop	Notes
Low Light (200 lux)	-6.2%	Use IR assist
Backlight (Window)	-8.5%	Silhouettes reduce accuracy
Fast Motion (<2 m/s)	-12.1%	Motion blur causes missed frames
Partial Occlusion	-15.3%	Wrist/hand partially out of frame
Multiple Hands	N/A	System ignores second hand
Gloved Hand (thin)	-4.7%	Landmarks still detectable
Gloved Hand (thick)	-28.9%	MediaPipe fails to extract joints

Recommendations: Best results will be achieved when using consistent front lighting and avoiding rapid left-right movements.

7. Applications

The ability to create a platform capable of being used in many different areas makes this an attractive project:

• Assistive Mobility: A gesture can control a wheelchair or hospital cart for people with limited hand function.

• STEM Education: The modular open source design lends itself very well to teaching students about robotics or IoT labs.

• AGV's in Industry: Using a gesture to "follow me" or to perform an emergency stop in a warehouse or industrial environment.

• Public Exhibit: Interactive demonstrations at a museum, science fair or technology fair that are safe for the public.

• Home Automation: Assistive robots for the elderly or disabled.

8. Conclusion & Future Work

We demonstrated an affordable, privacy-friendly and safe smart vehicle that is totally controlled using your hands through a series of gestures. The system combines MediaPipe (vision) technology with ESP32 and ultrasonic (embedded safety logic) to provide safe, reliable and easy-to-use control without compromising on users' private information as well as without requiring any wearable devices.

The total cost of components in the system is below Rs. 4500 (and therefore, very economical for academia). This provides the flexibility for the users to add to or modify the structure of this system.

Future Enhancements

- Replace the rules based classification tool to be a TensorFlow Lite model to allow for dynamic gesture functionality.
- Improve communication between the PC and glove by using a WebSocket with TLS to enable two-way telemetry of commands and telemetry data.
- Add an Inertial Measurement Unit (IMU) from the MPU6050 to provide the necessary data to perform inertial stabilization and to allow tilt-based control.
- Add a GPS receiver to allow the glove to navigate waypoints in an outside environment.
- Allow multiple hand tracking to create applications that are able to

track both hands simultaneously for use in collaborative or multi-user type applications.
- Create a companion application for Android and iOS to visualize the commands being sent to the glove as well as to allow for remote diagnostics.

References

[1] O. Kwon et al., "Natural user interfaces for robotics: a survey," IEEE Access 8 (2020): 123456-123467.

[2] J. LaViola, "A survey of hand posture and gesture recognition techniques," ACM Computing Surveys 38, no. 2 (2006): 1–30.

[3] M. Satyanarayanan, "The case for VM-based cloudlets in mobile computing," IEEE Pervasive Computing 8, no. 4 (2009): 14–23.

[4] A. Cavoukian, "Privacy by design: the 7 foundational principles," Information and Privacy Commissioner of Ontario (Canada), 2009.

[5] G. R. Bradski, "Real-time computer vision with OpenCV," IEEE Software 25, no. 3 (2008): 78–87.

[6] Y. Cui et al., "Deep learning for hand gesture recognition: a survey," Neurocomputing 468 (2022): 1–15.

[7] L. Zhang et al., "CNN-based real-time hand gesture recognition for human-robot interaction," Sensors 20, no. 18 (2020).

[8] Z. Ren et al., "Vision-based gesture control of mobile robot using YOLO and MediaPipe," in Proc. IEEE ICRA Workshop, 2021.

[9] S. Panchal et al., "EMG-based hand gesture recognition for prosthetic control," Biomedical Signal Processing and Control 68 (2021).

[10] J. Lee et al., "IMU glove interface for UAV navigation using dynamic time warping," IEEE Sensors Journal 19, no. 15 (2019).

[11] Y. Chen et al., "Latency analysis of cloud robotics architectures using AWS Lambda," IEEE IoT J. 7, no. 5 (2020): 3987–3998.

[12] M. Satyanarayanan, "Edge computing: vision and challenges," IEEE Internet of Things Journal 4, no. 5 (2017).

[13] A. Gupta et al., "Gesture controlled robot without obstacle avoidance: a case study of a student project," IJERT 5, no. 4 (2016).